Σύγκριση ταξινομητών 1-NN, 3-NN και NCC στο σύνολο εικόνων CIFAR-100

Ενδιάμεση εργασία — Νευρωνικά Δίκτυα και Βαθιά Μάθηση

***Περίληψη*---Η εργασία συγκρίνει τους ταξινομητές πλησιέστερου γείτονα με έναν και τρεις γείτονες (1-NN και 3-NN) με τον ταξινομητή πλησιέστερου κέντρου κλάσης (NCC). Χρησιμοποιείται το CIFAR-100, με 50 000 εικόνες εκπαίδευσης και 10 000 εικόνες ελέγχου, και ως χαρακτηριστικά διατηρούνται οι 3 072 τιμές RGB κάθε εικόνας. Παρουσιάζονται η οργάνωση των δεδομένων, η μετατροπή σε αριθμούς κινητής υποδιαστολής, η κλιμάκωση στο $[0, 1]$, οι κανόνες ταξινόμησης και η μέτρηση της ορθότητας. Η αρχική, απλή υλοποίηση διατηρείται ως αναφορά και συγκρίνεται με ξεχωριστή βελτιστοποιημένη υλοποίηση CPU. Η βελτιστοποίηση αφορά τον τρόπο εκτέλεσης των υπολογισμών και όχι την εισαγωγή νέων χαρακτηριστικών ή τη μείωση του συνόλου εκπαίδευσης. Στο πλήρες σύνολο ελέγχου οι ορθότητες είναι 17,55%, 17,48% και 10,04% αντίστοιχα. Η κοινή αναζήτηση γειτόνων για τα δύο k-NN ολοκληρώνεται σε 10,32 s στη μετρημένη εκτέλεση CPU.**

***Λέξεις κλειδιά*---CIFAR-100, ταξινόμηση εικόνων, k-NN, πλησιέστερο κέντρο κλάσης, ευκλείδεια απόσταση, ορθότητα, υπολογιστικό κόστος.**

I. Στόχος και πειραματική προσέγγιση

Στόχος είναι η σύγκριση τριών απλών κανόνων ταξινόμησης πάνω στα ίδια δεδομένα. Το ερώτημα είναι ποιο ποσοστό των εικόνων ελέγχου ταξινομεί σωστά κάθε κανόνας όταν χρησιμοποιούνται απευθείας οι τιμές των εικονοστοιχείων. Δεύτερο ερώτημα είναι πώς διαφέρει το υπολογιστικό κόστος ανάμεσα στην αποθήκευση όλων των παραδειγμάτων, όπως στο k-NN, και στη σύνοψή τους με ένα κέντρο ανά κλάση, όπως στο NCC.

Δεν εκπαιδεύεται νευρωνικό δίκτυο σε αυτό το παραδοτέο. Επίσης δεν εφαρμόζονται PCA, αύξηση δεδομένων ή εξαγωγή χαρακτηριστικών από προεκπαιδευμένο μοντέλο. Έτσι η σύγκριση αφορά συγκεκριμένα τους τρεις ταξινομητές στον ίδιο χώρο RGB. Οι τιμές $k = 1$ και $k = 3$ προκύπτουν από την εκφώνηση και δεν επιλέγονται ύστερα από δοκιμές στο σύνολο ελέγχου.

II. Το σύνολο δεδομένων και η αναπαράστασή του

Το CIFAR-100 περιλαμβάνει έγχρωμες εικόνες 32×32 εικονοστοιχείων, κατανεμημένες σε 100 κλάσεις. Σε κάθε κλάση αντιστοιχούν 500 εικόνες εκπαίδευσης και 100 εικόνες ελέγχου. Οι κλάσεις περιλαμβάνουν ζώα, φυτά, αντικείμενα, οχήματα και σκηνές. Χρησιμοποιούνται οι λεπτομερείς ετικέτες `fine_labels`, όχι οι 20 υπερκλάσεις [1], [2].

Η έκδοση Python διαθέτει αρχεία `train`, `test` και `meta`. Τα δύο πρώτα περιέχουν εικόνες και ετικέτες, ενώ το τρίτο αντιστοιχίζει τους αριθμητικούς κωδικούς στα ονόματα κλάσεων. Διατηρείται ο επίσημος διαχωρισμός: οι εικόνες ελέγχου δεν χρησιμοποιούνται ούτε για τον σχηματισμό των κέντρων ούτε ως υποψήφιοι γείτονες.

Πίνακας I
Πίνακες εισόδου και ετικετών

Μεταβλητή	Σχήμα NumPy	Περιεχόμενο
X_{train}	$(50\,000, 3\,072)$	Εικόνες εκπαίδευσης
y_{train}	$(50\,000,)$	Κλάση κάθε εικόνας
X_{test}	$(10\,000, 3\,072)$	Εικόνες ελέγχου
y_{test}	$(10\,000,)$	Πραγματικές κλάσεις

Α΄. Από την εικόνα στη γραμμή του πίνακα

Κάθε εικόνα έχει τρία κανάλια χρώματος, άρα

$$D = 32 \cdot 32 \cdot 3 = 3\,072. \tag{1}$$

Στο αρχείο δεδομένων οι τιμές έχουν ήδη μετατραπεί σε διάνυσμα: προηγούνται οι 1 024 τιμές του κόκκινου καναλιού, ακολουθούν οι 1 024 του πράσινου και έπειτα οι 1 024 του μπλε. Μία γραμμή του X επομένως είναι μία ολόκληρη εικόνα, ενώ μία στήλη αντιστοιχεί σε συγκεκριμένη θέση και κανάλι. Η τιμή y_i είναι η κλάση της ίδιας εικόνας στη γραμμή i. Το y είναι μονοδιάστατος πίνακας NumPy, όχι πίνακας με 100 στήλες ούτε κωδικοποίηση one-hot.

Για την προβολή αντιστρέφεται μόνο η διάταξη: η γραμμή μετασχηματίζεται αρχικά σε $(3, 32, 32)$ και στη συνέχεια σε $(32, 32, 3)$, όπως αναμένει η `imshow`. Η μετατροπή αυτή δεν συμμετέχει στην ταξινόμηση.

Β΄. Προεπεξεργασία

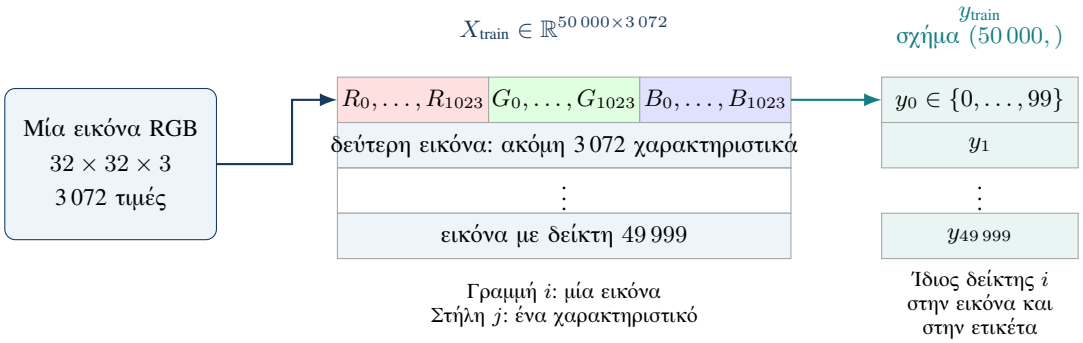
Οι αρχικές τιμές αποθηκεύονται ως `uint8` στο διάστημα $[0, 255]$. Πριν από τις αριθμητικές πράξεις μετατρέπονται σε `float32` και διαιρούνται με 255:

$$\widetilde{x}_{ij} = \frac{\text{float32}(x_{ij})}{255}. \tag{2}$$

Οι ετικέτες παραμένουν ακέραιοι αριθμοί. Η μετατροπή τύπου αποτρέπει την αναδίπλωση της μη προσημασμένης αριθμητικής κατά την αφαίρεση και επιτρέπει την αναπαράσταση κλασματικών τιμών. Η διαίρεση αλλάζει την κλίμακα, π.χ. $0 \mapsto 0$, $51 \mapsto 0{,}2$ και $255 \mapsto 1$.

Η ίδια συνάρτηση εφαρμόζεται στα δύο σύνολα. Δεν υπολογίζονται παράμετροι κλιμάκωσης από το test set: το 255 είναι το γνωστό μέγιστο της αρχικής αναπαράστασης. Σε ακριβή αριθμητική η κοινή αυτή κλιμάκωση διατηρεί τη σειρά των ευκλείδειων αποστάσεων, επειδή

$$d(\mathbf{x}/255, \mathbf{z}/255) = d(\mathbf{x}, \mathbf{z})/255. \tag{3}$$



Σχήμα 1. Η σύνδεση εικόνων και ετικετών. Η επιπέδωση αλλάζει την αναπαράσταση, όχι τις τιμές των εικονοστοιχείων. Για το σύνολο ελέγχου ισχύει η ίδια δομή με 10 000 γραμμές.

Άρα δεν αναμένεται από μόνη της βελτίωση της ορθότητας του k-NN ή του NCC.

Γ΄. Ομαδοποίηση κατά κλάση

Η αρχική υλοποίηση σχηματίζει το λεξικό `points`. Το κλειδί είναι ο αριθμός κλάσης και η τιμή ένας πίνακας με τις εικόνες της:

$$\texttt{points}[c] = X_{\text{train}}[y_{\text{train}} = c] \quad \text{με σχήμα } (500, 3\,072). \tag{4}$$

Η έκφραση $y = c$ δημιουργεί μία λογική μάσκα: επιλέγονται ακριβώς οι γραμμές που έχουν την αντίστοιχη ετικέτα. Η ομαδοποίηση χρησιμοποιεί μόνο το σύνολο εκπαίδευσης.

III. Οι τρεις ταξινομητές

Α΄. Ευκλείδεια απόσταση

Κοινό μέτρο ομοιότητας είναι η ευκλείδεια απόσταση δύο διανυσμάτων:

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{D} (x_j - z_j)^2}. \tag{5}$$

Κάθε χαρακτηριστικό έχει το ίδιο βάρος. Δεν υπάρχει ειδική μεταχείριση γειτονικών εικονοστοιχείων, περιγραμμάτων ή σχημάτων. Δύο εικόνες μπορεί να θεωρηθούν κοντινές λόγω παρόμοιου φόντου ή χρωματισμού, χωρίς να απεικονίζουν το ίδιο είδος αντικειμένου.

Β΄. 1-NN και 3-NN

Στο k-NN η κλήση `fit()` αποθηκεύει τα παραδείγματα εκπαίδευσης. Δεν υπολογίζονται βάρη και δεν εκτελείται επαναληπτική ελαχιστοποίηση σφάλματος. Για μία νέα εικόνα υπολογίζονται οι αποστάσεις από όλες τις εικόνες εκπαίδευσης και επιλέγεται το σύνολο $\mathcal{N}_k(\mathbf{x})$ των k κοντινότερων. Η πρόβλεψη είναι

$$\widehat{y} = \arg\max_{c} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]. \tag{6}$$

Το 1-NN αντιγράφει την κλάση του κοντινότερου παραδείγματος. Το 3-NN χρησιμοποιεί την πλειοψηφία των τριών κοντινότερων. Οι ψήφοι δεν σταθμίζονται με την απόσταση.

Στην αρχική έκδοση οι γείτονες ταξινομούνται πρώτα ως προς την απόσταση και έπειτα ως προς τον αριθμό κλάσης. Οι πίνακες εικόνων εξαιρούνται από το κλειδί ταξινόμησης, διότι η σύγκρισή τους σε ισοπαλία δεν δίνει μία απλή λογική τιμή. Αν στο 3-NN εμφανιστούν τρεις διαφορετικές κλάσεις, το `Counter` επιλέγει την πρώτη κλάση στη σειρά των γειτόνων που έχει το μέγιστο πλήθος ψήφων: συνήθως αυτή του κοντινότερου γείτονα, με την προηγούμενη σύμβαση στις ακριβείς ισοπαλίες αποστάσεων.

Γ΄. Πλησιέστερο κέντρο κλάσης (NCC)

Για κάθε κλάση c υπολογίζεται το κέντρο

$$\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i : y_i = c} \mathbf{x}_i, \qquad n_c = 500. \tag{7}$$

Η πράξη `mean(axis=0)` παίρνει τον μέσο όρο κάθε στήλης, δηλαδή κάθε θέσης και καναλιού, πάνω στις εικόνες της κλάσης. Παράγονται 100 διανύσματα μήκους 3 072. Η νέα εικόνα ταξινομείται ως

$$\widehat{y} = \arg\min_{c \in \{0,\ldots,99\}} d(\mathbf{x}, \boldsymbol{\mu}_c). \tag{8}$$

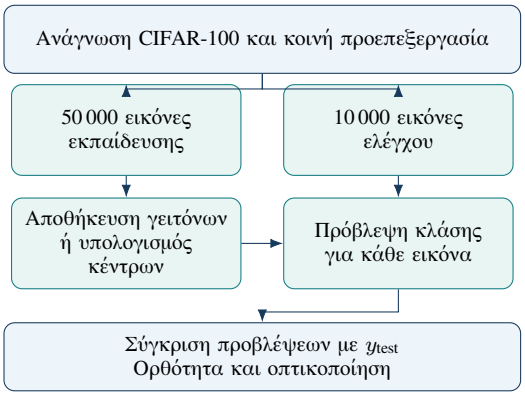
Ο NCC συγκρίνει συνεπώς μία εικόνα με 100 κέντρα αντί για 50 000 παραδείγματα. Η οικονομία αυτή έχει κόστος αναπαράστασης: διαφορετικές εμφανίσεις της ίδιας κλάσης συνοψίζονται σε έναν μοναδικό μέσο όρο. Σε ακριβή ισοπαλία επιλέγεται η πρώτη κλάση με την ελάχιστη απόσταση, με τις κλάσεις σε αύξουσα σειρά.

IV. Οργάνωση και βελτιστοποίηση του κώδικα

Α΄. Αρχική έκδοση

Το `src/main.py` χωρίζεται σε συναρτήσεις με διακριτές ευθύνες:

- `load_dataset`: ανάγνωση των αρχείων και ονομάτων κλάσεων.
- `preprocess_data`: μετατροπή τύπου, κλιμάκωση και ετικέτες.
- `group_by_class`: ομαδοποίηση μόνο των εικόνων εκπαίδευσης.
- `fit_classifiers`: δημιουργία και προσαρμογή των τριών μοντέλων.



Σχήμα 2. Διαχωρισμός της εκπαίδευσης από την αξιολόγηση. Οι αληθινές ετικέτες ελέγχου χρησιμοποιούνται για τη βαθμολόγηση των προβλέψεων.

- `evaluate_classifier`: προβλέψεις και μέτρηση ορθότητας.
- `plot_predictions`: εικόνες με πραγματική και προβλεπόμενη κλάση.

Η `main()` συνδέει τα στάδια. Η αξιολόγηση ολοκληρώνεται πριν ανοίξουν τα παράθυρα γραφημάτων και οι εικόνες χρησιμοποιούν ήδη αποθηκευμένες προβλέψεις. Έτσι δεν επαναλαμβάνεται η ταξινόμηση για την οπτικοποίηση.

Η αρχική αναζήτηση k-NN είναι εύκολη στην παρακολούθηση αλλά περιέχει έναν βρόχο Python για κάθε εικόνα εκπαίδευσης και πλήρη ταξινόμηση των αποστάσεων. Για ολόκληρο το test set, κάθε ανεξάρτητη εκτέλεση k-NN απαιτεί

$$50\,000 \cdot 10\,000 = 500\,000\,000 \tag{9}$$

συγκρίσεις εικόνων, με 3 072 χαρακτηριστικά ανά σύγκριση. Η εκτέλεση ξεχωριστά για $k = 1$ και $k = 3$ επαναλαμβάνει τον ίδιο υπολογισμό αποστάσεων. Η έκδοση αυτή είναι λειτουργική, αλλά ο μεγάλος χρόνος αποτέλεσε το κίνητρο για βελτιστοποίηση, όχι ένδειξη αποτυχίας του αλγορίθμου.

Β΄. Ξεχωριστή βελτιστοποιημένη έκδοση

Το αρχείο `src/optimized.py` περιέχει δύο νέες κλάσεις: `OptimizedKNearestNeighbors` και `OptimizedNearestClassCentroid`. Η αρχική έκδοση παραμένει ξεχωριστή. Δεν παραλείπονται εικόνες εκπαίδευσης και δεν εκτελείται προσεγγιστική αναζήτηση γειτόνων.

Για μία ομάδα B εικόνων ελέγχου, οι αποστάσεις υπολογίζονται με την ταυτότητα

$$\|\mathbf{x} - \mathbf{z}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{z}\|^2 - 2\mathbf{x}^{\mathsf{T}}\mathbf{z}. \tag{10}$$

Τα εσωτερικά γινόμενα υπολογίζονται με πολλαπλασιασμό πινάκων μέσω της βιβλιοθήκης BLAS. Τα τετράγωνα των νορμών των διανυσμάτων εκπαίδευσης υπολογίζονται μία φορά στη `fit()`. Δεν απαιτείται τετραγωνική ρίζα, επειδή είναι γνησίως αύξουσα συνάρτηση και δεν αλλάζει την κατάταξη.

Χρησιμοποιούνται ομάδες $B = 128$ εικόνων. Ο πίνακας αποστάσεων μίας ομάδας έχει σχήμα $128 \times 50\,000$ και καταλαμβάνει περίπου 51,2 MB με `float64`. Δεν σχηματίζεται τρισδιάστατος πίνακας διαφορών $B \times 50\,000 \times 3\,072$, ούτε αποθηκεύονται όλες οι αποστάσεις των 10 000 εικόνων ταυτόχρονα. Η διαίρεση σε ομάδες περιορίζει τη μνήμη· η επιτάχυνση προκύπτει από τις αριθμητικές πράξεις σε πίνακες και τη μείωση της εργασίας στους βρόχους Python.

Αντί για πλήρη ταξινόμηση 50 000 τιμών, η `partition` βρίσκει το όριο της k-οστής μικρότερης απόστασης. Επιλέγονται και ταξινομούνται μόνο οι υποψήφιοι έως αυτό το όριο, συμπεριλαμβανομένων όλων των ισοπαλιών στο όριο. Η τελική σειρά ορίζεται από απόσταση, κωδικό κλάσης και σειρά παραδείγματος. Οι τρεις κοντινότεροι γείτονες αρκούν για να παραχθούν μαζί οι προβλέψεις 1-NN και 3-NN, χωρίς δεύτερο πέρασμα αναζήτησης.

Για τον NCC διατηρείται ο ίδιος υπολογισμός μέσων όρων ανά κλάση και εφαρμόζεται η ίδια ταυτότητα αποστάσεων προς τα 100 κέντρα. Σε ισοπαλία διατηρείται η σειρά κλάσεων της αρχικής έκδοσης.

Οι είσοδοι παραμένουν κανονικοποιημένες τιμές `float32`, αλλά η βελτιστοποιημένη αριθμητική αποστάσεων γίνεται σε `float64`, ώστε να περιορίζεται το σφάλμα ακύρωσης της σχέσης (10). Μικρές αρνητικές υπολογισμένες τιμές αντικαθίστανται με μηδέν. Η μαθηματική απόσταση είναι ίδια, αλλά δεν διατυπώνεται εγγύηση ταυτόσημων bits με την αρχική αριθμητική· οι σχεδόν ισόπαλες αποστάσεις χρειάζονται έλεγχο. Για αυτό συγκρίνονται ρητά οι προβλέψεις των δύο εκδόσεων.

V. Αξιολόγηση και αποτελέσματα

Α΄. Πρωτόκολλο και μετρική

Όλοι οι ταξινομητές χρησιμοποιούν την ίδια αναπαράσταση και τις ίδιες ετικέτες. Η ορθότητα (accuracy) είναι

$$A = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbf{1}[\widehat{y}_i = y_i]. \tag{11}$$

Η μετρική αυτή απαντά ποιο ποσοστό των εικόνων αναγνωρίστηκε σωστά. Δεν είναι η ανά κλάση precision. Εδώ το test set είναι ισοκατανεμημένο στις 100 κλάσεις, οπότε η συνολική ορθότητα αποτελεί άμεση, εύληπτη βάση σύγκρισης. Ένας κανόνας που προβλέπει πάντα μία σταθερή κλάση πετυχαίνει ακριβώς 1% στο πλήρες σύνολο ελέγχου.

Οι χρόνοι πρόβλεψης μετρώνται χωριστά από φόρτωση αρχείων και γραφήματα. Το μικρό αρχικό πείραμα των 20 εικόνων χρησιμοποιήθηκε μόνο για έλεγχο της λειτουργίας και του κόστους. Δεν χρησιμοποιείται ως υποκατάστατο της τελικής σύγκρισης. Η σύγκριση ταχύτητας μεταξύ εκδόσεων απαιτεί ίδιο πλήθος εικόνων και ίδιο σύνολο εκπαίδευσης. Οι προεκτάσεις χρόνου σημειώνονται ως εκτιμήσεις, όχι ως πραγματικές πλήρεις εκτελέσεις.

Β΄. Μετρήσεις

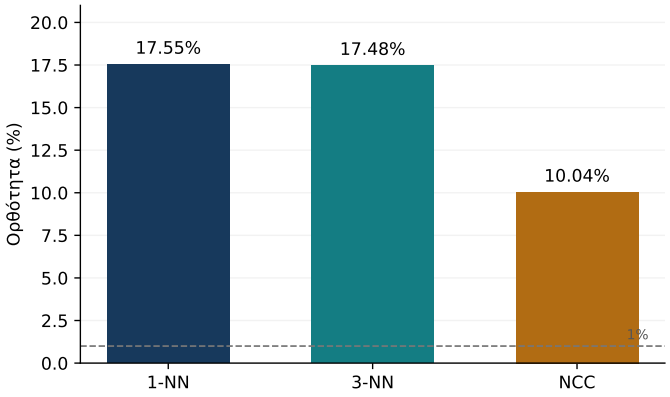
Οι μετρήσεις έγιναν σε CPU AMD Ryzen 9 7950X με οκτώ νήματα OpenBLAS, Python 3.10.12, NumPy 1.24.3 και OpenBLAS 0.3.21. Το μέγεθος ομάδας ήταν 128. Δεν χρησιμοποιήθηκε GPU. Οι χρόνοι είναι μία μέτρηση πραγματικού χρόνου ανά περίπτωση, μετά από σύντομη προθέρμανση,

Πίνακας II
Πλήρης αξιολόγηση: 10 000 εικόνες ελέγχου

Ταξινομητής	Σωστές προβλέψεις	Ορθότητα
1-NN	1 755	17,55%
3-NN	1 748	17,48%
NCC	1 004	10,04%

Πίνακας III
Χρόνος πρόβλεψης στο ίδιο δείγμα 100 εικόνων

Μοντέλο	Αρχική (s)	Βελτ. (s)	Λόγος
1-NN	24,173	0,0916	$263,8\times$
3-NN	24,031	0,1095	$219,5\times$
NCC	0,0417	0,000589	$70,9\times$



Σχήμα 3. Μετρημένη ορθότητα των βελτιστοποιημένων εκδόσεων στο πλήρες test set. Η διακεκομμένη γραμμή δείχνει το 1% ενός σταθερού προβλεπόμενου κωδικού κλάσης στο ισοκατανεμημένο σύνολο.

και εξαρτώνται από το μηχάνημα και τον ταυτόχρονο φόρτο του. Δεν παρουσιάζονται ως μέσοι όροι πολλών επαναλήψεων.

Για σύγκριση των δύο εκδόσεων επιλέχθηκε μία εικόνα ελέγχου από κάθε κλάση, με γεννήτρια `default_rng` και σπόρο 2026: συνολικά 100 εικόνες. Κάθε ταξινομητής χρησιμοποίησε και τις 50 000 εικόνες εκπαίδευσης. Στο κοινό αυτό δείγμα κάθε βελτιστοποιημένη έκδοση συμφώνησε με την αντίστοιχη αρχική σε **100 από 100 προβλέψεις**. Η συμφωνία αυτή ελέγχει το δείγμα, όχι κάθε δυνατή αριθμητική ισοπαλία. Για τον NCC εκτελέστηκε επιπλέον και η αρχική έκδοση σε ολόκληρο το test set, με συμφωνία 10 000/10 000.

Ο Πίνακας III αφορά ξεχωριστές κλήσεις πρόβλεψης ανά ταξινομητή. Στην τελική εκτέλεση η κοινή αναζήτηση για 1-NN και 3-NN ολοκληρώθηκε σε **10,319 s** για όλες τις 10 000 εικόνες· ο χρόνος αυτός ανήκει *και στα δύο αποτελέσματα μαζί*, όχι σε κάθε μοντέλο χωριστά. Η πρόβλεψη NCC χρειάστηκε 0,0567 s. Η αρχική NCC χρειάστηκε 4,158 s στο ίδιο πλήρες σύνολο.

Το `fit()` της απλής k-NN αποθηκεύει μόνο μία αναφορά στο λεξικό και μετρήθηκε περίπου στο 1 μs. Η βελτιστοποιημένη κοινή προετοιμασία k-NN χρειάστηκε 1,523 s, επειδή δημιουργεί συνεχόμενο πίνακα `float64`, ετικέτες και τετράγωνα νορμών. Αυτή η προετοιμασία πληρώνεται μία φορά και δεν περιλαμβάνει τη φόρτωση ή την ομαδοποίηση. Ο σχηματισμός κέντρων NCC χρειάστηκε περίπου 0,023 s και στις δύο εκδόσεις.

Δεν εκτελέστηκαν οι δύο αρχικές k-NN στο πλήρες test set. Η αναγωγή των χρόνων του δείγματος δίνει περίπου 40 λεπτά ανά αρχικό πέρασμα· πρόκειται για *εκτίμηση*, όχι πλήρη μέτρηση. Η αρχική υλοποίηση είναι λειτουργική. Η μετρημένη επιτάχυνση CPU κατέστησε περιττή την προσθήκη GPU ή PCA για το παρόν πείραμα.

Τα αρχεία `results/cpu_benchmark.json` και `results/predictions.npz` περιέχουν αντίστοιχα τη ρύθμιση, τις μετρήσεις και τις προβλέψεις στην αρχική σειρά του test set. Το JSON καταγράφει επίσης τους δείκτες του δείγματος και αποτυπώματα SHA-256 των πηγαίων αρχείων της μετρημένης εκτέλεσης.

Γ΄. Παραδείγματα ταξινόμησης

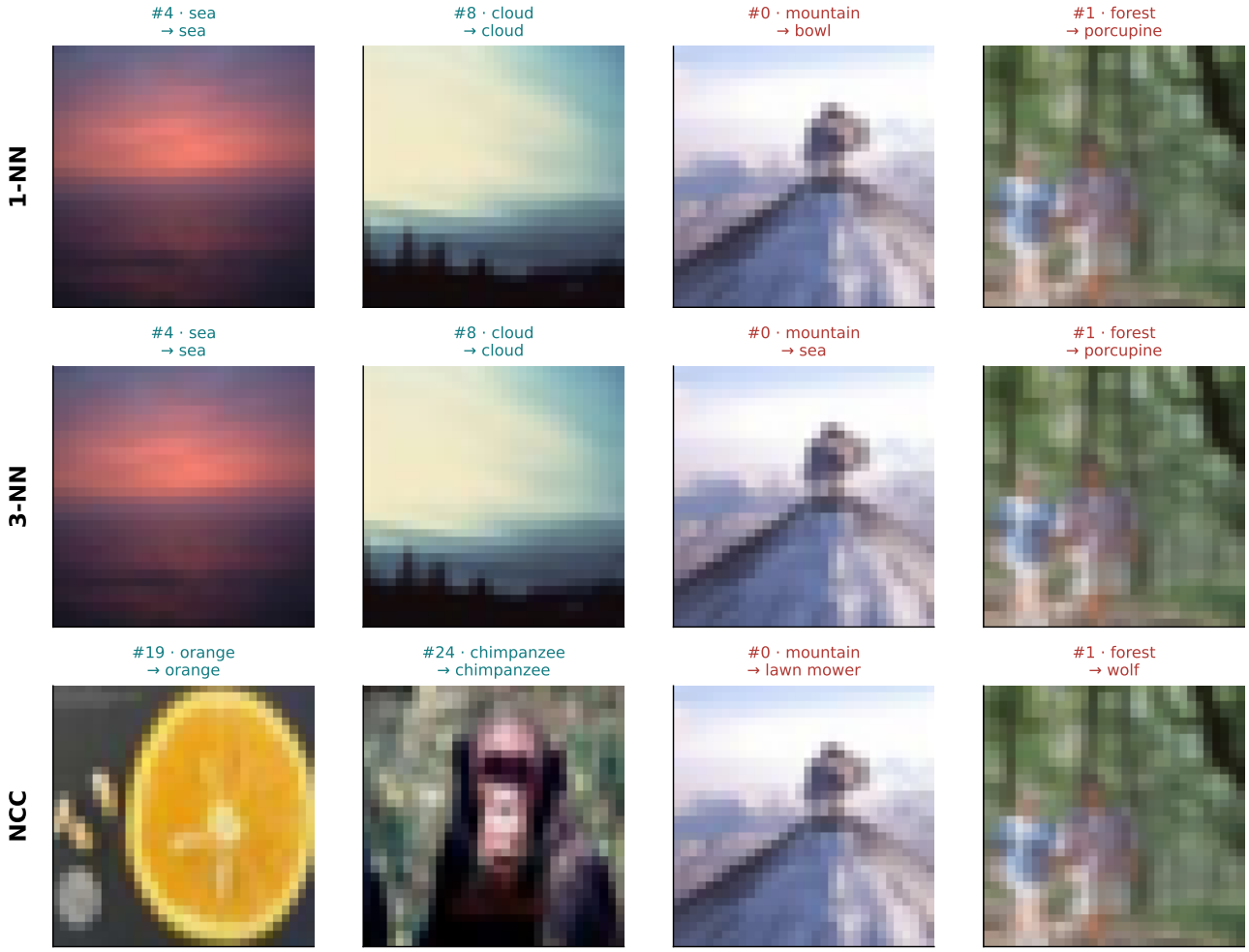
Το Σχήμα 4 δείχνει πραγματικές προβλέψεις από την πλήρη εκτέλεση. Για κάθε ταξινομητή επιλέγονται, με τη σειρά των δεικτών του test set, οι πρώτες δύο σωστές και οι πρώτες δύο λανθασμένες προβλέψεις. Η επιλογή είναι αναπαραγώγιμη και δεν αποτελεί τυχαίο ή αντιπροσωπευτικό δείγμα της συνολικής απόδοσης. Τα ονόματα παραμένουν στην αγγλική μορφή του CIFAR-100 για άμεση αντιστοίχιση με τα μεταδεδομένα.

VI. Συζήτηση και περιορισμοί

Η ευκλείδεια απόσταση πάνω σε ακατέργαστες τιμές RGB δεν παρέχει αναλλοιωτότητα ως προς μετατόπιση, προσανατολισμό ή αλλαγές φωτισμού. Η μικρή ανάλυση επίσης περιορίζει την οπτική πληροφορία. Έτσι, ακόμη και ένα αντικείμενο που είναι εύκολα αναγνωρίσιμο από άνθρωπο δεν είναι υποχρεωτικά κοντά στα παραδείγματα της ίδιας κλάσης στον συγκεκριμένο διανυσματικό χώρο.

Το 3-NN μπορεί να περιορίσει την επιρροή ενός μεμονωμένου γείτονα, αλλά δεν υπάρχει εγγύηση ότι θα ξεπεράσει το 1-NN: οι πρόσθετοι γείτονες μπορεί να ανήκουν σε άλλη κλάση. Ο NCC μειώνει δραστικά τις συγκρίσεις κατά την πρόβλεψη, αλλά ένα μόνο κέντρο δεν αποτυπώνει πολλαπλές ομάδες εμφανίσεων της ίδιας κλάσης. Η μετρημένη σύγκριση αφορά αυτόν τον συνδυασμό δεδομένων, χαρακτηριστικών και απόστασης και όχι μία γενική κατάταξη των αλγορίθμων.

Τα δεδομένα είναι ένα συγκεκριμένο σύνολο εικόνων αναφοράς και δεν τεκμηριώνουν απόδοση σε πραγματικές εφαρμογές ή σε εικόνες άλλης προέλευσης. Δεν πραγματοποιήθηκε ανεξάρτητος έλεγχος κάθε εικόνας για πιθανές ομοιότητες μεταξύ training και test. Διατηρήθηκε ο επίσημος διαχωρισμός και δεν προστέθηκαν δεδομένα από προσωπικές συλλογές. Η διάθεση των αρχικών εικόνων διέπεται από τους όρους της πηγής τους· δεν συνάγεται γενική άδεια επαναχρησιμοποίησης από τη δυνατότητα λήψης του συνόλου.



Σχήμα 4. Μία σειρά ανά ταξινομητή. Οι δύο αριστερές στήλες είναι σωστές προβλέψεις (πράσινοι τίτλοι), οι δύο δεξιές λανθασμένες (κόκκινοι τίτλοι). Πάνω από κάθε εικόνα αναγράφονται ο δείκτης στο test set και η πραγματική κλάση· μετά το βέλος ακολουθεί η πρόβλεψη.

VII. Συμπεράσματα

Στο συγκεκριμένο πείραμα το 1-NN πέτυχε 17,55%, το 3-NN 17,48% και ο NCC 10,04%. Τα δύο k-NN βρίσκονται πολύ κοντά: η διαφορά είναι μόλις επτά σωστές προβλέψεις, δηλαδή 0,07 ποσοστιαίες μονάδες. Χωρίς περαιτέρω στατιστική ανάλυση δεν τεκμηριώνεται γενική υπεροχή του 1-NN. Και τα δύο ξεπέρασαν τον NCC σε αυτόν τον χώρο χαρακτηριστικών, ενώ ο NCC είχε πολύ χαμηλότερο κόστος πρόβλεψης.

Η διάκριση αλγορίθμου και υλοποίησης ήταν ουσιαστική: η αρχική εκδοχή έκανε σαφή τη λειτουργία των κανόνων, ενώ η νέα αξιοποίησε πράξεις πινάκων, περιορισμένη επιλογή γειτόνων και κοινή αναζήτηση για δύο τιμές του k. Η συμφωνία με την αναφορά ελέγχθηκε πριν χρησιμοποιηθούν τα τελικά αποτελέσματα. Η πλήρης κοινή αναζήτηση k-NN ολοκληρώθηκε σε περίπου 10,3 s σε CPU, χωρίς αλλαγή συνόλου εκπαίδευσης ή εφαρμογή PCA. Η σχετικά χαμηλή ορθότητα υπενθυμίζει ότι η ταχύτερη εκτέλεση δεν προσθέτει σημασιολογική πληροφορία στις αρχικές τιμές RGB.

Παράρτημα Α′
Αρχική υλοποίηση: κρίσιμα αποσπάσματα

Τα παρακάτω αποσπάσματα προέρχονται απευθείας από το συνοδευτικό `src/main.py`. Το πλήρες αρχείο περιλαμβάνει επίσης φόρτωση, προεπεξεργασία, εκτέλεση και γραφήματα.

```python
def euclidean_distance(p, q):
    return np.sqrt(np.sum((np.array(p) - np.array(q)) ** 2))
```

```python
    def k_nearest(self, new_point):
        """Return the k closest training points as (distance, category, point)."""
        distances = []

        for category, class_points in self.points.items():
            for point in class_points:
                distance = euclidean_distance(point, new_point)
                distances.append((distance, category, point))

        # Exclude image arrays from comparisons when distance and category tie.
        return sorted(distances, key=lambda item: (item[0], item[1]))[: self.k]

    def predict(self, new_point):
        categories = [category for _, category, _ in self.k_nearest(new_point)]
        return Counter(categories).most_common(1)[0][0]
```

```python
    def fit(self, points):
        # Example for this dataset: {"blue": [2.0, 2.6], "red": [4.8, 5.4]}
        self.centroids = {
            category: np.mean(class_points, axis=0)
            for category, class_points in points.items()
        }

    def predict(self, new_point):
        return min(
            self.centroids,
            key=lambda category: euclidean_distance(
                self.centroids[category], new_point
            ),
        )
```

```python
def evaluate_classifier(classifier, X_test, y_test):
    """Return predictions and accuracy for an already fitted classifier."""
    if len(X_test) == 0:
        raise ValueError("The evaluation set must contain at least one image.")
    if len(X_test) != len(y_test):
        raise ValueError("Each evaluation image must have one label.")

    predictions = []
    correct_predictions = 0

    for i in range(len(X_test)):
        prediction = classifier.predict(X_test[i])
        predictions.append(prediction)
        if prediction == y_test[i]:
            correct_predictions += 1

    accuracy = correct_predictions / len(X_test)
    return np.array(predictions), accuracy
```

Παράρτημα Β′
Βελτιστοποιημένη υλοποίηση: κρίσιμα αποσπάσματα

Τα παρακάτω αποσπάσματα προέρχονται από το συνοδευτικό `src/optimized.py`. Η κοινή συνάρτηση χρησιμοποιείται τόσο για παραδείγματα εκπαίδευσης όσο και για κέντρα κλάσεων.

```python
def squared_distances(queries, references, reference_norms):
    """Use ||x-y||² = ||x||² + ||y||² − 2 x·y without a 3D temporary."""
    distances = queries @ references.T
    distances *= -2.0
    distances += np.einsum("ij,ij->i", queries, queries)[:, None]
    distances += reference_norms[None, :]
    # Cancellation can produce tiny negative values for coincident points.
    np.maximum(distances, 0.0, out=distances)
    return distances
```

```python
        for start in range(0, len(X), self.batch_size):
            stop = min(start + self.batch_size, len(X))
            distances = squared_distances(X[start:stop], self.X, self.norms)
            for offset, row in enumerate(distances):
                threshold = np.partition(row, k - 1)[k - 1]
                # Include all boundary ties before ordering by distance/category/index.
                candidates = np.flatnonzero(row <= threshold)
                order = np.lexsort((candidates, self.labels[candidates], row[candidates]))
                neighbors[start + offset] = candidates[order[:k]]
            if progress:
                print(f"k-NN search: {stop}/{len(X)} images", flush=True)
        return neighbors
```

```python
    def predict_for_k(self, X, ks=(1, 3), progress=False):
        """Reuse one exhaustive search for several values of k."""
        if not ks or any(not isinstance(k, int) or k < 1 for k in ks):
            raise ValueError("Provide positive integer neighbor counts.")
        indices = self.kneighbors(X, k=max(ks), progress=progress)
        labels = self.labels[indices]
        return {
            k: np.array([
                Counter(row[:k]).most_common(1)[0][0] for row in labels
            ], dtype=self.labels.dtype)
            for k in ks
        }
```

Στον βελτιστοποιημένο NCC η πρόβλεψη προς τα κέντρα γίνεται με την ίδια κοινή συνάρτηση αποστάσεων:

```python
    def predict_many(self, X):
        if self.centroids is None:
            raise ValueError("Fit the classifier before predicting.")
        X = query_array(X, self.centroids.shape[1])
        predictions = np.empty(len(X), dtype=self.labels.dtype)
        for start in range(0, len(X), self.batch_size):
            stop = min(start + self.batch_size, len(X))
            distances = squared_distances(X[start:stop], self.centroids, self.norms)
            predictions[start:stop] = self.labels[np.argmin(distances, axis=1)]
        return predictions
```

Η `src/benchmark.py` εκτελεί τον έλεγχο συμφωνίας, τη χρονομέτρηση και την πλήρη αξιολόγηση. Τα πλήρη αρχεία παρέχονται μαζί με την έκθεση.

Αναφορές

[1] A. Krizhevsky, V. Nair και G. Hinton, ``CIFAR-10 and CIFAR-100 datasets.'' https://www.cs.toronto.edu/~kriz/cifar.html.

[2] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, τεχνική αναφορά, University of Toronto, 2009, κεφ. 3. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[3] Εκφώνηση πρώτης υποχρεωτικής εργασίας, μάθημα «Νευρωνικά Δίκτυα -- Βαθιά Μάθηση», ενότητα «Ενδιάμεση Εργασία», 2026.